

Contents

1	Examples	5
1.1	Write and read a string to/from an <code>iostream</code>	5
1.2	Write and read a <code>map<string,string></code> to/from an <code>iostream</code>	6
1.3	Turtles all the way down	7
1.4	Integrating a user defined type	8
2	Primary I/O Classes	9
2.1	<code>Dwm::StreamIO</code>	10
2.1.1	Using <code>Dwm::StreamIO</code> with user defined types	10
2.2	<code>Dwm::FileIO</code>	11
2.2.1	Using <code>Dwm::FileIO</code> with user defined types	11
2.3	<code>Dwm::DescriptorIO</code>	12
2.3.1	Using <code>Dwm::DescriptorIO</code> with user defined types	12
2.4	<code>Dwm::BZ2IO</code>	13
2.4.1	Using <code>Dwm::BZ2IO</code> with user defined types	13
2.5	<code>Dwm::GZIO</code>	14
2.5.1	Using <code>Dwm::GZIO</code> with user defined types	14
2.6	<code>Dwm::ASIO</code>	14
3	Directly supported data types	15
3.1	integral types	15
3.2	floating point types	15
3.3	enumerated types	15
3.4	<code>std::string</code>	15
3.5	<code>std::vector<bool></code>	15
4	Supported <code>std</code> type templates	16
4.1	<code>std::array<T,N></code>	16

4.2	<code>std::deque<T></code>	16
4.3	<code>std::list<T></code>	16
4.4	<code>std::map<KeyType, MappedType></code>	16
4.5	<code>std::multimap<KeyType, MappedType></code>	16
4.6	<code>std::optional<T></code>	16
4.7	<code>std::pair<FirstType, SecondType></code>	16
4.8	<code>std::set<T></code>	16
4.9	<code>std::multiset<T></code>	16
4.10	<code>std::unique_ptr<T></code>	17
4.11	<code>std::unordered_map<KeyType, MappedType></code>	17
4.12	<code>std::unordered_multimap<KeyType, MappedType></code>	17
4.13	<code>std::unordered_set<T></code>	17
4.14	<code>std::unordered_multiset<T></code>	17
4.15	<code>std::vector<T></code>	17
5	Concepts	18
5.1	Streamable	18
5.2	C++ iostreams	18
5.2.1	<code>HasStreamRead<T></code>	18
5.2.2	<code>HasStreamWrite<T></code>	18
5.2.3	<code>Dwm::IsStreamReadable<T></code>	18
5.2.4	<code>Dwm::IsStreamWritable<T></code>	18
5.3	FILE pointer (cstdio)	19
5.3.1	<code>HasFileRead<T></code>	19
5.3.2	<code>HasFileWrite<T></code>	19
5.4	UNIX descriptors	19
5.4.1	<code>HasDescriptorRead<T></code>	19
5.4.2	<code>HasDescriptorWrite<T></code>	19
5.5	Boost asio sockets	20
5.5.1	<code>HasAsioRead<T></code>	20

<i>CONTENTS</i>	3
5.5.2 HasAsioWrite<T>	20
5.6 BZFILE pointer (bzip2)	20
5.6.1 HasBZRead<T>	20
5.6.2 HasBZWrite<T>	20
5.7 gzFile (libz)	20
5.7.1 HasGZRead<T>	20
5.7.2 HasGZWrite<T>	20
5.8 HasSkipAnnotation<std::meta::info info>	20
5.9 Streamed length	21
5.9.1 Constant streamed length	21
5.9.2 Constant streamed length via static constexpr uint64_t StreamedLength() member	21
5.9.3 Constant streamed length from C++26 reflection	21
6 On-the-wire formats	22
6.1 On-the-wire formats, network byte order (MSB first)	22
6.1.1 16 bit integral types (int16_t, uint16_t)	22
6.1.2 32 bit integral types (int32_t, uint32_t)	22
6.1.3 64 bit integral types (int64_t, uint64_t)	22
6.1.4 float	22
6.1.5 double	22
6.1.6 std::string	23
6.1.7 std::pair<T1,T2>	23
6.1.8 std::vector<T>	23
6.1.9 std::deque<T>	23
6.1.10 std::list<T>	23
6.1.11 std::map<Key,Value>	24
6.1.12 std::multimap<Key,Value>	24
6.1.13 std::unordered_map<Key,Value>	24
6.1.14 std::unordered_multimap<Key,Value>	24
6.1.15 std::set<T>	24

6.1.16	<code>std::multiset<T></code>	25
6.1.17	<code>std::unordered_set<T></code>	25
6.1.18	<code>std::unordered_multiset<T></code>	25

Chapter 1

Examples

1.1 Write and read a string to/from an iostream

The program below writes a `std::string` to a `std::stringstream`, then reads a `std::string` from the `std::stringstream`. The program in listing 1.1 returns 0 if the string we read matches the string we wrote.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using std::string, std::stringstream;
7     bool      success{false};
8     string     writeString("Hello, world!"), readString;
9     stringstream ss;
10    if (Dwm::StreamIO::Write(ss, writeString)) {
11        if (Dwm::StreamIO::Read(ss, readString)) {
12            success = (readString == writeString);
13        }
14    }
15    return success ? 0 : 1;
16 }
```

Listing 1.1: Write and read a string to/from an iostream

1.2 Write and read a `map<string,string>` to/from an `iostream`

The program in listing 1.2 writes a `std::map<std::string,std::string>` to a `std::stringstream`, then reads a `std::map<std::string,std::string>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers holding supported types (`std::string` is an explicitly supported type).

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using stringmap = std::map<std::string,std::string>;
7     stringmap m1 {
8         { "apple",    "fruit"    },
9         { "cat",      "feline"   },
10        { "broccoli", "vegetable" },
11        { "dog",       "canine"   },
12        { "kiwi",      "fruit"    },
13        { "cow",       "bovine"   }
14    };
15    bool success{false};
16    std::stringstream ss;
17    if (Dwm::StreamIO::Write(ss, m1)) {
18        stringmap m2;
19        if (Dwm::StreamIO::Read(ss, m2)) {
20            success = (m2 == m1);
21        }
22    }
23    return success ? 0 : 1;
24 }
```

Listing 1.2: Write and read a `map<string,string>` to/from an `iostream`

1.3 Turtles all the way down

The program in listing 1.3 writes a `std::map<std::string, std::vector<std::string>>` to a `std::stringstream`, then reads a `std::map<std::string, std::vector<std::string>>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers and supported types being “turtles all the way down”: our support is recursive to any depth.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using container = std::map<std::string, std::vector<std::string>>;
7     container m1 {
8         { "fruit",      { "apple", "banana", "kiwi", "orange" } },
9         { "canine",     { "husky", "labrador", "poodle", "wolf", "collie" } },
10        { "feline",      { "cougar", "lion", "panther" } },
11        { "vegetable", { "broccoli", "carrot", "squash" } }
12    };
13
14    bool success{false};
15    std::stringstream ss;
16    if (Dwm::StreamIO::Write(ss, m1)) {
17        container m2;
18        if (Dwm::StreamIO::Read(ss, m2)) {
19            success = (m2 == m1);
20        }
21    }
22    return success ? 0 : 1;
23 }
```

Listing 1.3: Write and read a `map<string, vector<string>>` to/from an `iostream`

1.4 Integrating a user defined type

The program in listing 1.4 utilizes a user defined type `MyType` that has `Dwm::StreamIO` functionality. `MyType` meets the requirements of the `HasStreamRead<T>` and `HasStreamWrite<T>` concepts by providing the `Read()` member at line 9 and the `Write()` member at line 12.

```

1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 //-----
5 struct MyType {
6     uint32_t    id;
7     std::string name;
8
9     std::istream & Read(std::istream & is)
10    { return Dwm::StreamIO::ReadV(is, id, name); }
11
12    std::ostream & Write(std::ostream & os) const
13    { return Dwm::StreamIO::WriteV(os, id, name); }
14
15    bool operator <=> (const MyType &) const = default;
16 };
17
18 //-----
19 int main(int argc, char *argv[])
20 {
21     bool                rc{false};
22     std::vector<MyType> myvec1 { {1, "cat"}, {2, "dog" }, { 3, "fish" } };
23     std::stringstream    ss;
24     if (Dwm::StreamIO::Write(ss, myvec1)) {
25         std::vector<MyType> myvec2;
26         if (Dwm::StreamIO::Read(ss, myvec2)) {
27             rc = (myvec2 == myvec1);
28         }
29     }
30     return rc ? 0 : 1;
31 }

```

Listing 1.4: Integrating a user defined type

The main function creates a vector of `MyType` named `myvec1` on line 22. It writes `myvec1` to the `stringstream` named `ss` at line 24. It then creates a second vector of `MyType` named `myvec2` on line 25 and reads the contents of `myvec2` from `ss` on line 26. On line 27 it checks if the vector we read (`myvec2`) matches the vector we wrote (`myvec1`).

Chapter 2

Primary I/O Classes

There are several primary I/O classes in libDwm. Each is used with a particular type of source and sink.

`Dwm::StreamIO` is used with C++ iostreams (`std::istream` and `std::ostream`).

`Dwm::FileIO` is used with `FILEs` (`<cstdio>`).

`Dwm::DescriptorIO` is used with UNIX file descriptors.

`Dwm::BZ2IO` is used with bz2 streams (BZFILES opened via `BZ2_bzopen()`).

`Dwm::GZIO` is used with gz streams (gzFiles opened via `gzopen()`).

When adding I/O support for your own data type (struct or class), `Dwm::StreamIO` support would typically be the first target. That's largely because for fixed length types, all of the other I/O classes can be supported via an intermediate pass through `Dwm::StreamIO` with a `std::stringstream` as a data buffer.

2.1 Dwm::StreamIO

source type: `std::istream`

sink type: `std::ostream`

`Dwm::StreamIO` provides static functions and function templates to read and write from/to C++ iostreams. Conceptually, the primary interface looks like four static member function templates:

```
namespace Dwm {
    class StreamIO {
    public:
        template <typename T>
        static std::istream & Read(std::istream & is, T & t);

        template <typename T>
        static std::ostream & Write(std::ostream & os, const T & t);

        template <typename... Args>
        static std::istream & ReadV(std::istream & is, Args & ...args);

        template <typename... Args>
        static std::ostream & WriteV(std::ostream & os, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for `Read()` and `Write()`. It can't automatically read and write any type `T`; a data type must either be directly supported or meet the requirements of `Dwm::HasStreamRead` and `Dwm::HasStreamWrite`.

Note that `ReadV()` and `WriteV()` are just convenient variadic function templates that will read or write multiple objects in sequence.

2.1.1 Using `Dwm::StreamIO` with user defined types

If a user-defined type provides the following two public members, `Dwm::StreamIO` will utilize them to read and write instances of the user defined type:

```
std::istream & Read(std::istream & is)
std::ostream & Write(std::ostream & os) const
```

See the `Dwm::HasStreamRead` and `Dwm::HasStreamWrite` concepts for the semantic requirements of these members.

2.2 Dwm::FileIO

source type: FILE *

sink type: FILE *

Dwm::FileIO provides static functions and function templates to read and write from/to FILES. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class FileIO {
    public:
        template <typename T>
        static size_t Read(FILE *f, T & t);

        template <typename T>
        static size_t Write(FILE *f, const T & t);

        template <typename... Args>
        static size_t ReadV(FILE *f, Args & ...args);

        template <typename... Args>
        static size_t WriteV(FILE *f, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#).

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

2.2.1 Using Dwm::FileIO with user defined types

If a user-defined type provides the following two public members, Dwm::FileIO will utilize them to read and write instances of the user defined type:

```
size_t Read(FILE *f)
size_t Write(FILE *f) const
```

See the [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#) concepts for the semantic requirements of these members.

2.3 Dwm::DescriptorIO

source type: int
sink type: int

Dwm::DescriptorIO provides static functions and function templates to read and write from/to UNIX file descriptors. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class DescriptorIO {
    public:
        <template typename T> static ssize_t Read(int fd, T & t);

        <template typename T> static ssize_t Write(int fd, const T & t);

        template <typename ...Args>
        static ssize_t ReadV(int fd, Args & ...args);

        template <typename ...Args>
        static ssize_t WriteV(int fd, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasDescriptorRead](#) and [Dwm::HasDescriptorWrite](#).

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

2.3.1 Using Dwm::DescriptorIO with user defined types

If a user defined type provides the following two public members, Dwm::DescriptorIO will utilize them to read and write instances of the user defined type:

```
ssize_t Read(int fd)
ssize_t Write(int fd) const
```

See the [Dwm::HasDescriptorRead](#) and [Dwm::HasDescriptorWrite](#) concepts for the semantic requirements of these members.

2.4 Dwm::BZ2IO

source type: BZFILE *

sink type: BZFILE *

Dwm::BZ2IO provides static functions and function templates to read and write from/to bz2 streams (BZFILE *). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class BZ2IO {
    public:
        <template typename T>
        static int BZRead(BZFILE *bzf, T & t);

        <template typename T>
        static int BZWrite(BZFILE *bzf, const T & t);

        template <typename ...Args>
        static int BZReadV(BZFILE *bzf, Args & ...args);

        template <typename ...Args>
        static int BZWriteV(BZFILE *bzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for BZRead() and BZWrite(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#).

2.4.1 Using Dwm::BZ2IO with user defined types

If a user defined type provides the following two public members, Dwm::BZ2IO will utilize them to read and write instances of the user defined type:

```
int BZRead(BZFILE *)
int BZWrite(BZFILE *) const
```

See the [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#) concepts for the semantic requirements of these member functions.

2.5 Dwm::GZIO

gzFile source type: gzFile
 sink type: gzFile

Dwm::GZIO provides static functions and function templates to read and write from/to gz streams (gzFile). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class GZIO {
    public:
        <template typename T>
        static int Read(gzFile gzf, T & t);

        <template typename T>
        static int Write(gzFile gzf, const T & t);

        template <typename ...Args>
        static int ReadV(gzFile gzf, Args & ...args);

        template <typename ...Args>
        static int WriteV(gzFile gzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#).

2.5.1 Using Dwm::GZIO with user defined types

If a user defined type provides the following two public members, Dwm::GZIO will utilize them to read and write instances of the user defined type:

```
int Read(gzFile)
int Write(gzFile) const
```

See the [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#) concepts for the expected semantics of these members.

2.6 Dwm::ASIO

Boost asio sockets

Chapter 3

Directly supported data types

Directly supported data types are those types for which we have static member functions to read the type from a source and write the type to a sink. These types include integral types, floating point types, enumerated types, `std::string` and `std::vector<bool>`.

3.1 integral types

The category of types for which `std::is_integral_v<T>` evaluates to `true`.

For multi-byte types in this category, we use an MSB-first representation on-the-wire.

3.2 floating point types

The category of types for which `std::is_floating_point_v<T>` evaluates to `true`.

We use XDR for the on-the-wire representation of `float` and `double`.

3.3 enumerated types

The category of types for which `std::is_enum_v<T>` evaluates to `true`.

3.4 `std::string`

3.5 `std::vector<bool>`

Chapter 4

Supported **std** type templates

4.1 **std::array<T,N>**

We can stream any instances of `std::array<T,N>` where T is streamable.

4.2 **std::deque<T>**

We can stream any instances of `std::deque<T>` where T is streamable.

4.3 **std::list<T>**

We can stream any instances of `std::list<T>` where T is streamable.

4.4 **std::map<KeyType,MappedType>**

We can stream any instances of `std::map<KeyType,MappedType>` where KeyType and MappedType are streamable.

4.5 **std::multimap<KeyType,MappedType>**

We can stream any instances of `std::multimap<KeyType,MappedType>` where KeyType and MappedType are streamable.

4.6 **std::optional<T>**

We can stream any instances of `std::optional<T>` where T is streamable.

4.7 **std::pair<FirstType,SecondType>**

4.8 **std::set<T>**

We can stream any instances of `std::set<T>` where T is streamable.

4.9 **std::multiset<T>**

We can stream any instances of `std::multiset<T>` where T is streamable.

4.10 `std::unique_ptr<T>`

We can stream instances of `std::unique_ptr<T>` where `T` is streamable and the `unique_ptr`'s `deleter_type` is `std::default_delete<T>`. We use the `deleter_type` test to infer that the `std::unique_ptr<T>` points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

4.11 `std::unordered_map<KeyType, MappedType>`

We can stream any instances of `std::unordered_map<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.12 `std::unordered_multimap<KeyType, MappedType>`

We can stream any instances of `std::unordered_multimap<KeyType, MappedType>` where `KeyType` and `MappedType` are streamable.

4.13 `std::unordered_set<T>`

We can stream any instances of `std::unordered_set<T>` where `T` is streamable.

4.14 `std::unordered_multiset<T>`

We can stream any instances of `std::unordered_multiset<T>` where `T` is streamable.

4.15 `std::vector<T>`

We can stream any instances of `std::vector<T>` where `T` is streamable.

Chapter 5

Concepts

5.1 Streamable

5.2 C++ iostreams

5.2.1 HasStreamRead<T>

Evaluates to true if T has a member function with signature:

```
std::istream & Read(std::istream & is)
```

Required semantics

- Must populate *this with contents read from is
- Must return is

Other semantics

- May set std::ios_base::failbit

5.2.2 HasStreamWrite<T>

Evaluates to true if T has a member function with signature:

```
std::ostream & Write(std::ostream & os) const
```

Required semantics

- Must write the contents of *this to os
- Must return os

Other semantics

- May set std::ios_base::failbit

5.2.3 Dwm::IsStreamReadable<T>

5.2.4 Dwm::IsStreamWritable<T>

5.3 FILE pointer (cstdio)

5.3.1 HasFileRead<T>

Evaluates to true if T has a member function with signature:

`size_t Read(FILE *f)`

Required semantics

- Must populate *this with contents read from f
- Must return a non-zero size_t on success, 0 on failure

5.3.2 HasFileWrite<T>

Evaluates to true if T has a member function with signature:

`size_t Write(FILE *f) const`

Required semantics

- Must write the contents of *this to f
- Must return a non-zero size_t on success, 0 on failure

5.4 UNIX descriptors

5.4.1 HasDescriptorRead<T>

Evaluates to true if T has a member function with signature:

`ssize_t Read(int fd)`

Required semantics

- Must populate *this with contents read from fd
- Must return a positive ssize_t on success, -1 on failure

5.4.2 HasDescriptorWrite<T>

Evaluates to true if T has a member function with signature:

`ssize_t Write(int fd) const`

Required semantics

- Must write the contents of *this to fd
- Must return a positive ssize_t on success, -1 on failure

5.5 Boost asio sockets

5.5.1 HasAsioRead<T>

5.5.2 HasAsioWrite<T>

5.6 BZFILE pointer (bzip2)

5.6.1 HasBZRead<T>

Evaluates to true if T has a member function with signature:

```
int BZRead(BZFILE *bzf)
```

Required semantics

- Must populate *this with contents read from bzf
- Must return a positive int on success, -1 on failure

5.6.2 HasBZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(BZFILE *bzf) const
```

Required semantics

- Must write the contents of *this to bzf
- Must return a positive int on success, -1 on failure

5.7 gzFile (libz)

5.7.1 HasGZRead<T>

Evaluates to true if T has a member function with signature:

```
int Read(gzFile gzf)
```

Required semantics

- Must populate *this with contents read from gzf
- Should return the number of decompressed bytes read on success, -1 on failure

5.7.2 HasGZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(gzFile gzf) const
```

Required semantics

- Must write the contents of *this to gzf
- Should return the number of uncompressed bytes written on success, -1 on failure

5.8 HasSkipAnnotation<std::meta::info info>

Evaluates to true if the given reflection info has an annotation of the form `[ [=Dwm::skip_io]]`

This requires C++26 reflection, specifically P2996 and P3394.

5.9 Streamed length

5.9.1 Constant streamed length

5.9.2 Constant streamed length via `static constexpr uint64_t StreamedLength()` member

5.9.3 Constant streamed length from C++26 reflection

When using C++26 reflection for serialization and deserialization of instances of a given type, we can determine at compile time whether or not the streamed length of the type is constant, and hence whether or not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

Chapter 6

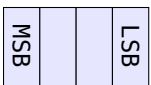
On-the-wire formats

6.1 On-the-wire formats, network byte order (MSB first)

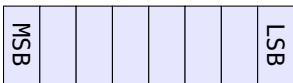
6.1.1 16 bit integral types (`int16_t`, `uint16_t`)



6.1.2 32 bit integral types (`int32_t`, `uint32_t`)

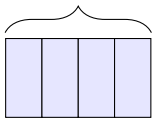


6.1.3 64 bit integral types (`int64_t`, `uint64_t`)



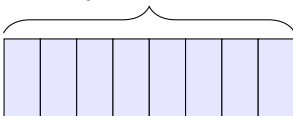
6.1.4 **float**

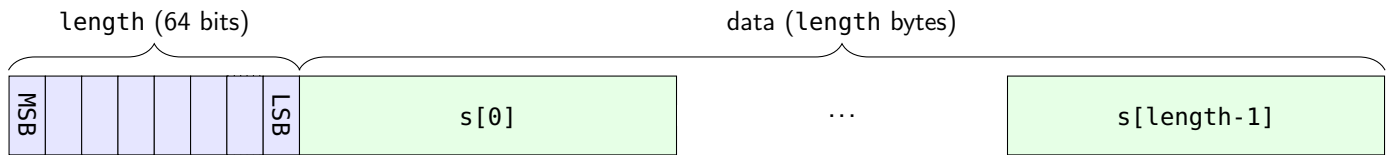
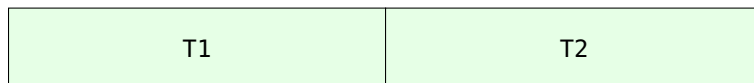
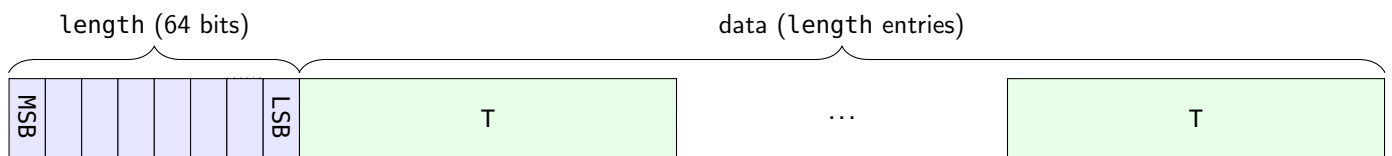
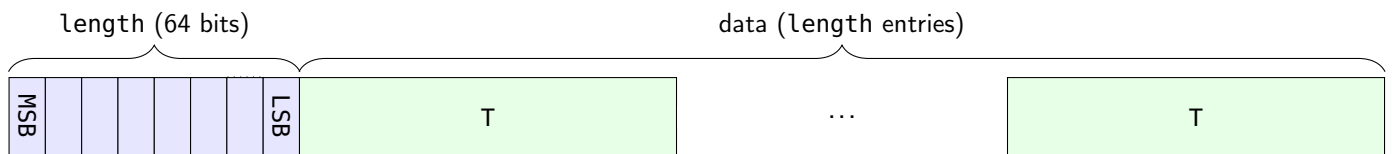
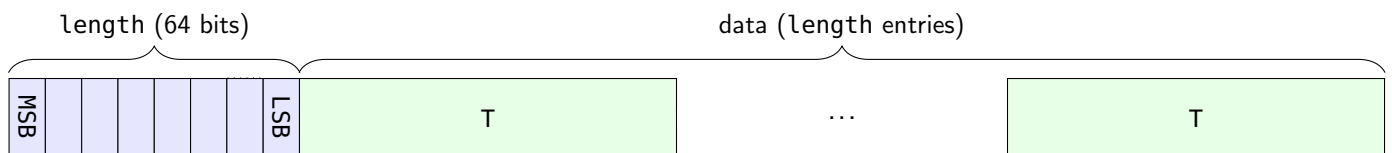
4 bytes, RFC4506

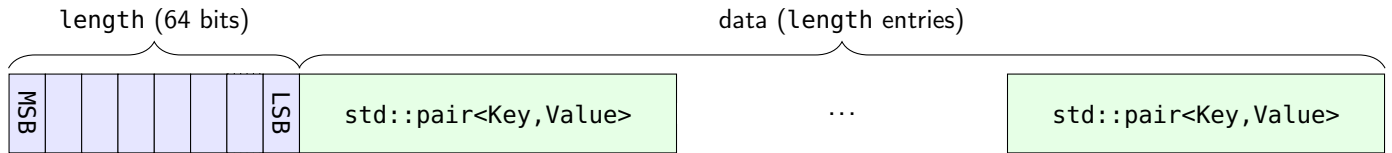
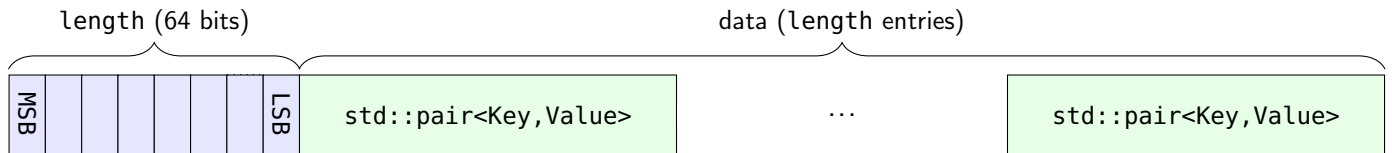
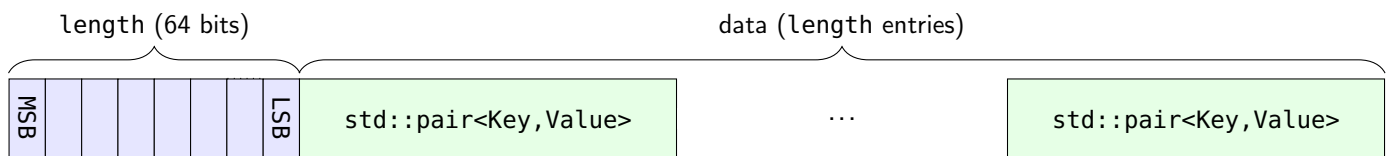
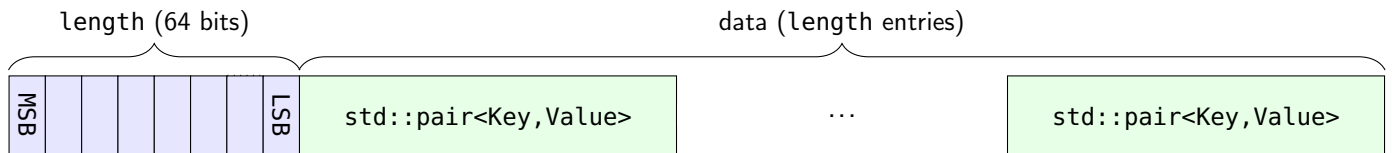
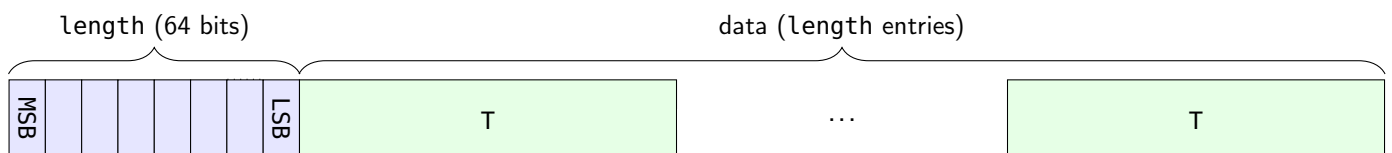


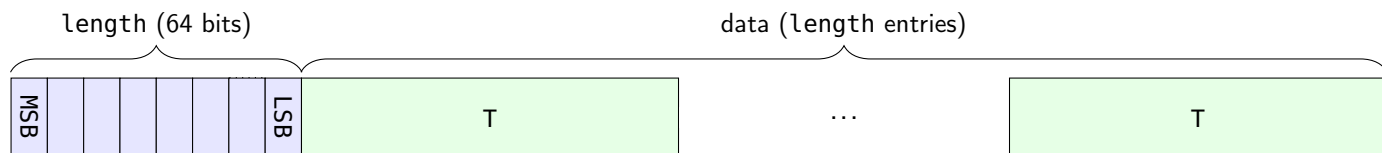
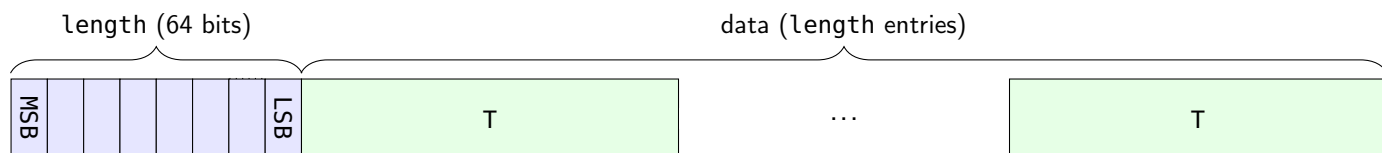
6.1.5 **double**

8 bytes, RFC4506



6.1.6 `std::string`**6.1.7 `std::pair<T1,T2>`****6.1.8 `std::vector<T>`****6.1.9 `std::deque<T>`****6.1.10 `std::list<T>`**

6.1.11 `std::map<Key, Value>`**6.1.12 `std::multimap<Key, Value>`****6.1.13 `std::unordered_map<Key, Value>`****6.1.14 `std::unordered_multimap<Key, Value>`****6.1.15 `std::set<T>`**

6.1.16 `std::multiset<T>`**6.1.17 `std::unordered_set<T>`****6.1.18 `std::unordered_multiset<T>`**